International University of Grand-Bassam (IUGB)

Department of Computer Science

Design and Implementation of an AI-Enhanced Restaurant Management System with Personalized Recommendations, Natural Language Processing, and Demand Forecasting

Capstone Final Project Report

Bachelor of Science Capstone Project

Prepared as part of the requirements for the Bachelor of Science degree

Academic Year: 2025-2026

Declaration

I hereby declare that this capstone project report is my original work and that all sources used in the preparation of this report have been acknowledged in the reference section. The software system described in this report was designed and implemented as part of the requirements for the Bachelor of Science capstone project at the International University of Grand-Bassam.

Student signature: _____

Date: _____

Dedication

I dedicate this work to my family, mentors, instructors, and everyone who supported me throughout my academic journey. Their encouragement, patience, and guidance helped me complete this project and continue developing as a software engineer.

Acknowledgements

I would like to express my sincere gratitude to the International University of Grand-Bassam, the Department of Computer Science, and my capstone instructor for their guidance during the development of this project. I am also grateful to my classmates, friends, and family for their support, feedback, and encouragement throughout the implementation and writing process.

Abstract

This report presents the design and implementation of a full-stack restaurant management system enhanced with three artificial intelligence modules: a personalized recommendation engine, a Natural Language Processing chatbot assistant, and a moving-average demand forecasting service. The system is implemented with a Vue.js 3 Single-Page Application frontend, a Node.js and Express.js RESTful backend, and a MySQL relational database. The project addresses common limitations in restaurant software, including passive menu discovery, fragmented customer interaction, and limited administrative visibility into future demand. The recommendation module combines user order history, food category/type preferences, global popularity, and a cold-start fallback. The chatbot combines intent classification with rule-based menu filtering and food-name resolution. The forecasting module estimates future food demand from recent completed or active orders using a moving-average velocity approach. Formal deployment testing, real restaurant user studies, and large-scale forecasting accuracy measurements were outside the scope of this capstone.

Table of Contents

- Chapter 1 - Introduction
- Chapter 2 - Restaurant Management Systems
- Chapter 3 - Artificial Intelligence Techniques Used in the Project
- Chapter 4 - System Analysis and Design
- Chapter 5 - System Implementation
- Chapter 6 - System Evaluation
- Chapter 7 - Conclusion and Future Work
- References
- Appendices

Note: Page numbers should be updated in Microsoft Word before final PDF export if an automatic table of contents is inserted.

List of Figures

- Figure 4.1 - Three-tier system architecture
- Figure 4.2 - Entity-relationship overview of the restaurant database
- Figure 4.3 - Checkout and order persistence data flow
- Figure 5.1 - Hybrid recommendation engine processing pipeline
- Figure 5.2 - NLP chatbot layered resolution pipeline
- Figure 5.3 - Demand forecasting endpoint output
- Figure 5.4 - Floating chatbot user interface with food cards
- Figure 6.1 - Precision@K for the recommendation engine using repository seed data
- Figure 6.2 - Recall@K for the recommendation engine using repository seed data
- Figure 6.3 - Precision and recall comparison across recommendation cutoffs
- Figure 6.4 - Distribution of chatbot training examples by intent
- Figure 6.5 - Forecast confidence distribution based on activity-day regularity

List of Tables

- Table 1.1 - Project objectives and verification methods
- Table 1.2 - Definition of terms
- Table 4.1 - Functional requirements
- Table 4.2 - Non-functional requirements
- Table 4.3 - Database schema overview
- Table 4.4 - Main API domains
- Table 6.1 - Functional validation summary
- Table 6.2 - Recommendation holdout metrics computed from repository seed data
- Table 6.3 - Chatbot training utterance distribution
- Table 6.4 - Forecast confidence distribution from repository seed data
- Table 6.5 - Highest seed-data forecast outputs using the moving-average velocity formula

- Table 6.6 - Evaluation limitations and recommended validation improvements

Chapter 1 - Introduction

1.1 Background and Motivation

The food service industry has undergone a major digital transformation. Customers increasingly expect to browse menus online, place orders without physical queues, track order progress, and receive assistance before making a purchase decision. Restaurants, in turn, need tools that help them manage orders, payments, table bookings, customer records, and administrative decisions from a unified system.

Many restaurant platforms provide the basic ability to list menu items and record orders. However, such systems often remain passive: they store transactions but do not use the available data to support customer discovery or administrative planning. The motivation for this project is to demonstrate how a conventional restaurant management platform can be strengthened with practical artificial intelligence features while remaining understandable, maintainable, and appropriate for an undergraduate capstone project.

1.2 Problem Statement

Standard restaurant management platforms present several limitations. First, customers often browse large menus manually, even when their previous order history could be used to surface more relevant items. Second, customers may have simple questions about menu categories, dietary preferences, spicy foods, or dish pairings, but most basic systems do not provide conversational assistance. Third, restaurant administrators commonly rely on intuition to anticipate demand, even though historical order data can provide useful signals about future item popularity.

The problem addressed in this project is therefore the design and implementation of a restaurant management system that supports core ordering and administration tasks while also using available data to provide recommendations, natural language menu assistance, and simple demand estimates.

1.3 Project Objectives

The system is designed to achieve the following specific goals:

Table 1.1 - Project objectives and verification methods

#	Objective	Verification Method
O1	Implement a RESTful API that supports CRUD operations for menu items, user accounts, carts, bookings, and orders.	Manual API and frontend workflow testing
O2	Implement a hybrid recommendation engine that blends user order history with	Endpoint validation and offline evaluation route

#	Objective	Verification Method
	global popularity and fallback logic.	
O3	Implement a co-purchase recommendation module based on items purchased in the same bill.	SQL co-occurrence query validation
O4	Implement a restaurant chatbot that handles greetings, recommendations, dish information, menu filters, and pairings.	Representative functional chatbot queries
O5	Implement an administrative demand forecasting endpoint using recent order history.	Forecast endpoint validation against seeded transaction data
O6	Integrate the AI modules into a customer-facing Vue.js interface and administrator dashboard.	Frontend workflow validation

1.4 Scope and Limitations

The project scope includes a full-stack web application, a relational database, customer ordering workflows, account creation and login, cart management, checkout, table booking, order tracking, an administrator dashboard, a recommendation service, a chatbot service, and a demand forecasting endpoint.

The project does not include native mobile applications, real payment gateway processing, WebSocket-based real-time notifications, production deployment benchmarking, or a controlled user study in an operating restaurant. The chatbot is trained on a fixed domain-specific set of examples and supported by rule-based matching. The demand forecast is based on a moving-average approach rather than a complex statistical or deep learning forecasting model.

1.5 Definition of Terms

Table 1.2 - Definition of terms

Term	Definition
SPA	Single-Page Application: a web application that loads a single HTML shell and dynamically changes views in the browser.
REST	Representational State Transfer: an architectural style that organizes web APIs around resources and HTTP verbs.
NLP	Natural Language Processing: a branch of AI concerned with enabling computers to process human language.
Recommendation System	A system that ranks or suggests items to users based on behavior, item attributes, or both.
Cold Start	A recommendation problem where a new user or item has little or no historical interaction data.
Precision@K	The proportion of the top K recommendations that are relevant.
Recall@K	The proportion of relevant holdout items recovered in the top K recommendations.
Demand Forecasting	The process of estimating future demand from historical sales or order data.

1.6 Report Organization

Chapter 1 introduces the motivation, problem statement, objectives, scope, limitations, and terminology of the project. Chapter 2 discusses restaurant management systems as the application domain. Chapter 3 presents the artificial intelligence concepts used in the system, including recommendation systems, NLP chatbots, and demand forecasting. Chapter 4 explains the system analysis and design, including requirements, architecture, database design, API design, user roles, and data flow. Chapter 5 describes the actual implementation of the frontend, backend, database, recommendation engine, chatbot, forecasting service, and ordering workflow. Chapter 6 evaluates the implemented system through functional testing and module-level validation. Chapter 7 concludes the report and proposes future work.

Chapter 2 - Restaurant Management Systems

2.1 Definition of Restaurant Management Systems

A restaurant management system is an information system designed to support the operational, administrative, and customer-facing activities of a restaurant. Such a system may include menu management, ordering, table reservation, billing, payment tracking, inventory planning, customer records, reporting, and staff coordination. From an operations perspective, these activities are closely related to inventory and production planning because restaurant managers must align expected demand with available ingredients and labor capacity (Silver et al., 2017).

2.2 Evolution of Digital Restaurant Platforms

Restaurant technology has evolved from simple cash registers and paper tickets to integrated digital platforms. Early systems focused mainly on point-of-sale operations, while modern systems often include online ordering, delivery integration, customer profiles, digital menus, kitchen status tracking, and administrative dashboards. This evolution is consistent with the broader development of single-page web applications, where client-side interfaces can support complex interactive workflows without constant full-page reloads (Mikowski & Powell, 2013).

2.3 Online Ordering Systems

Online ordering systems allow customers to browse available foods, choose quantities, place items in a cart, provide delivery or pickup information, and submit an order. The ordering process also creates behavioral data. Each order can be interpreted as a signal of customer preference and item demand, making online ordering the data foundation for recommendations and forecasting.

2.4 Inventory and Order Management

Inventory and order management are central problems in restaurant operations. Restaurants must prepare enough ingredients to satisfy demand while avoiding excessive waste. Although the implemented system does not include a complete inventory module, the demand forecasting feature is related to the same operational concern: estimating which food items are likely to be requested in a future period.

2.5 Benefits of Restaurant Information Systems

Restaurant information systems centralize menu data, reduce repeated manual entry, preserve customer and order records, improve administrative visibility, support faster customer service, and create the historical data necessary for analytics. In this project, the same order records support checkout, order tracking, recommendations, co-purchase suggestions, and demand analysis.

2.6 Challenges in Modern Restaurant Operations

Modern restaurant operations face challenges such as fluctuating demand, menu complexity, customer indecision, dietary preferences, order tracking, and administrative workload. The present capstone

responds to these challenges by adding focused AI features rather than attempting to automate the entire restaurant.

2.7 Relevance to the Proposed System

The theoretical concepts discussed in this chapter explain why the project combines operational workflows with AI-supported features. A restaurant management system must first manage core data correctly: foods, users, carts, bookings, orders, and payments. Once these workflows are in place, the stored data can support recommendations and demand estimates.

Chapter 3 - Artificial Intelligence Techniques Used in the Project

This chapter presents the artificial intelligence concepts that guided the project. The discussion focuses on three techniques directly implemented in the system: recommendation systems, natural language processing for chatbot interaction, and demand forecasting for administrative planning.

3.1 Recommendation Systems

3.1.1 Definition

A recommendation system is a software component that ranks or suggests items to users based on available signals. The recommender systems literature identifies user behavior, item attributes, and hybrid combinations as common sources of recommendation evidence (Ricci et al., 2015). In this project, recommendations help restaurant customers discover foods that may match their preferences.

3.1.2 Collaborative Filtering

Collaborative filtering recommends items based on patterns of interaction. Item-to-item recommendation has been widely used in commercial recommender systems because relationships between items can be easier to maintain than direct user-to-user similarity at scale (Linden et al., 2003). The project uses a simplified item co-purchase idea in the Users Also Bought module, where foods ordered in the same bill are treated as related; this is conceptually connected to association-rule mining, which identifies frequent co-occurrence patterns in transaction data (Agrawal & Srikant, 1994).

3.1.3 Content-Based Filtering

Content-based filtering recommends items with attributes similar to items a user has liked or purchased. The implemented personalized recommender uses category and food type preferences derived from the user order history, which makes it partially content-based.

3.1.4 Hybrid Recommendation Systems

Hybrid recommendation systems combine multiple signals to reduce the weaknesses of any single technique. Large-scale production systems such as Netflix have shown the value of combining multiple recommendation signals and business constraints rather than relying on a single model family (Gomez-Uribe & Hunt, 2016). The implemented recommendation engine follows this general idea at a simpler capstone scale by combining user history, food category/type preference, global popularity, vote information, and fallback strategies.

3.1.5 Cold Start Problem

The cold start problem occurs when a system has insufficient historical data about a user or item. The implemented system addresses this by falling back to popular foods for guests and new users, and by blending popularity with limited preference data for low-history users.

3.1.6 Evaluation Metrics: Precision@K and Recall@K

Recommendation systems are commonly evaluated with ranking metrics such as Precision@K and Recall@K. Herlocker et al. (2004) emphasize that recommender evaluation must match the decision task, which is why top-K ranking metrics are useful when the system presents a short list of suggestions.

The project includes an evaluation endpoint that applies a simple holdout split when a user has enough unique historical items. Because the available transaction data is limited and seeded for development, these metrics should be interpreted as functional evaluation support rather than evidence of production-level recommendation quality.

3.2 Natural Language Processing and Chatbots

3.2.1 Definition of NLP

Natural Language Processing is a branch of artificial intelligence concerned with enabling computers to process, analyze, and respond to human language (Jurafsky & Martin, 2023).

3.2.2 Intent Classification

Intent classification maps a user utterance to a predefined category such as greeting, recommendation request, dish information request, spicy food request, or vegan food request. This follows the common task-oriented dialogue pattern in which a system first identifies the user's intent before selecting a domain-specific response (Jurafsky & Martin, 2023). The chatbot uses node-nlp to classify short English utterances into restaurant-specific intents.

3.2.3 Rule-Based Chatbots

Rule-based chatbots use predefined patterns, dictionaries, and conditional logic to respond to user inputs. The project uses rule-based matching for category aliases, food names, cuisine types, meat preferences, vegan preferences, spice level, and price preference.

3.2.4 Machine Learning Chatbots

Machine learning chatbots learn intent or response behavior from labeled examples. Modern transformer architectures significantly expanded the capabilities of language models (Vaswani et al., 2017), but the implemented chatbot does not use a transformer or large language model. Instead, it uses a small machine-learning intent classifier through node-nlp and does not claim open-ended conversational ability.

3.2.5 Domain-Specific Chatbots

A domain-specific chatbot is designed for a limited task area. The chatbot in this project is limited to restaurant menu support, including recommendations, food descriptions, category filtering, spicy/vegan/meat requests, and co-purchase pairings.

3.2.6 NLP in Customer Service

NLP can improve customer service by answering frequent questions, reducing navigation effort, and guiding users toward relevant items. In this project, the chatbot provides assistance inside the customer-facing web interface and can return food cards that allow direct cart interaction.

3.3 Demand Forecasting

3.3.1 Definition of Demand Forecasting

Demand forecasting is the process of estimating future demand using historical data. In operations and inventory management, demand estimates are used to plan purchasing, production, and replenishment decisions (Silver et al., 2017). In restaurants, demand forecasting can help administrators prepare ingredients, anticipate popular items, and reduce waste.

3.3.2 Importance in Restaurants

Restaurant demand is affected by customer preferences, time, promotions, seasonality, and menu availability. Even a simple forecast can be useful when administrators need a quick indication of which items have been ordered frequently in a recent window.

3.3.3 Forecasting Techniques

Several forecasting techniques are available. Moving average methods estimate future demand from recent historical averages. Exponential smoothing gives greater weight to recent observations. ARIMA models capture autoregressive and moving-average patterns in time series. Machine learning approaches can use many features but require more data, tuning, and evaluation. The choice of method should be aligned with data availability, operational purpose, and the cost of model complexity (Silver et al., 2017).

3.3.4 Moving Average

A moving average is simple, interpretable, and easy to implement. It is appropriate when the objective is to provide a baseline forecast from limited historical data. Its weakness is that it may not capture seasonality, sudden changes, day-of-week patterns, or external events.

3.3.5 Exponential Smoothing

Exponential smoothing improves on a basic average by assigning more weight to recent observations. This can react faster to changing demand, but it requires selecting smoothing parameters and still needs a meaningful history of observations.

3.3.6 ARIMA

ARIMA models are classical statistical models for time series forecasting. They can model temporal dependence but require stationarity considerations, parameter selection, and enough time-indexed data.

3.3.7 Machine Learning Approaches

Machine learning forecasting models can incorporate features such as day of week, promotions, price, weather, holidays, or user segments. Their strength is flexibility, but their weakness is dependence on larger datasets and careful validation.

3.3.8 Engineering Justification for Moving Average

A moving-average approach was selected because it is transparent, easy to validate against seeded transactions, and consistent with the limited data available in the prototype. The goal was not to prove

that moving average is superior to other techniques. The goal was to implement a practical baseline forecast that administrators can interpret and that future work can replace or compare against more advanced models.

3.3.9 Demand Forecasting Model Selection and Justification

Restaurants benefit from demand forecasting because their operations depend on anticipating uncertain customer demand. A restaurant must purchase ingredients before customers order, schedule workers before service begins, and prepare menu items under time constraints. If demand is underestimated, the restaurant may experience stockouts, longer waiting times, and lower customer satisfaction. If demand is overestimated, the restaurant may purchase too many perishable ingredients and increase food waste.

Demand forecasting is closely related to inventory management. Historical sales quantities can help managers estimate how much of each ingredient or menu item may be needed for a future period. In food service, this relationship is especially important because many ingredients are perishable. A useful forecast may assist managers in planning purchases more carefully, reducing unnecessary inventory accumulation, and avoiding situations where popular items become unavailable.

Forecasting can also support staffing decisions. If the system indicates that demand for several high-preparation items may increase, managers could potentially schedule more kitchen or delivery support for that period. Conversely, if demand appears low, managers may choose a lighter staffing plan. In this capstone, staffing optimization is not implemented as a module; however, the forecast output can provide a data signal that could inform such decisions in future versions.

The project selected a moving-average velocity model for practical engineering reasons. The available dataset was small, the system was designed as an undergraduate capstone prototype, and the goal was to create an interpretable baseline rather than a complex black-box model. Moving average requires low computational resources, can be implemented directly with SQL aggregation and simple arithmetic, and produces outputs that administrators can understand without statistical training.

This choice should not be interpreted as a claim that moving average is the best forecasting method. More advanced methods may outperform it when large historical datasets, seasonality, promotions, holidays, and external variables are available. In this project, moving average was the most practical choice because it matched the data scale, implementation time, explainability requirement, and deployment simplicity of the system.

3.3.10 Comparison with Alternative Forecasting Approaches

Exponential smoothing is a common extension of moving average. Its advantage is that it can give more weight to recent observations, allowing the forecast to respond faster to changes in demand. Its disadvantage is that it requires selecting smoothing parameters, and with limited data those parameters may be unstable or arbitrary. This method was not selected because the project prioritized a simpler baseline that could be explained and validated directly from aggregate quantities.

ARIMA models are useful for time series that contain autocorrelation patterns and enough ordered observations. Their advantage is that they provide a formal statistical framework for modeling temporal dependence. Their disadvantages include the need for stationarity checks, parameter selection,

diagnostic analysis, and a longer time-indexed dataset. ARIMA was not selected because the project did not contain sufficient real restaurant time-series data to justify a formal ARIMA modeling process.

Machine learning forecasting methods can combine many predictors, such as day of week, menu category, discounts, weather, holidays, customer segments, and previous sales. Their advantage is flexibility. Their disadvantage is that they require larger datasets, feature engineering, model training, and careful validation to avoid overfitting. These methods were not selected because the capstone dataset and project timeline were not appropriate for a reliable supervised learning forecasting study.

Deep learning forecasting methods, including LSTM models and time-series transformers, can model complex nonlinear patterns in large sequential datasets. Their advantage is expressive power when large amounts of data are available. Their disadvantages include high computational cost, reduced interpretability, longer development time, and the need for substantial training data. These methods were not selected because they would add complexity without sufficient evidence that the available restaurant data could support them.

The comparison above is conceptual only. No experiment was conducted comparing moving average, exponential smoothing, ARIMA, machine learning models, or deep learning models. A fair comparison would require a larger historical dataset, consistent train/test splits, and forecasting error metrics such as MAE, RMSE, or MAPE.

Chapter 4 - System Analysis and Design

4.1 System Requirements

The system requirements were derived from the restaurant workflows and AI-enhancement goals discussed in the previous chapters. Requirements were divided into functional and non-functional categories to separate what the system must do from how well it should operate.

4.2 Functional Requirements

Table 4.1 - Functional requirements

ID	Requirement	Implemented Evidence
FR1	Customers shall be able to browse and filter menu items.	Implemented through Menu.vue, Vuex food state, and GET /api/foods.
FR2	Customers shall be able to register, log in, and maintain cart state.	Implemented through user and cart routes.
FR3	Customers shall be able to place orders and track order status.	Implemented through checkout, billdetails, billstatus, and MyOrder.vue.
FR4	Customers shall be able to reserve tables.	Implemented through Table.vue and POST /api/booking.
FR5	The system shall provide personalized and popular menu recommendations.	Implemented through GET /api/recommendations/:id.
FR6	The system shall provide a chatbot assistant for menu questions.	Implemented through POST /api/chatbot/message.
FR7	Administrators shall be able to view and update order status.	Implemented through Admin.vue and Dashboard.vue.
FR8	Administrators shall be able to view demand forecasts.	Implemented through GET /api/billstatus/forecast.

4.3 Non-Functional Requirements

Table 4.2 - Non-functional requirements

Category	Requirement
Usability	The web interface should allow customers to complete menu browsing, cart updates, checkout, booking, and order tracking without command-line interaction.
Maintainability	Frontend, backend, route, controller, model, and SQL responsibilities should remain separated so that features can be changed independently.
Scalability	The prototype uses a REST API and relational schema that can be extended, although production-scale load testing was outside the capstone scope.
Security	The prototype identifies important security gaps, including plaintext passwords and missing backend authorization middleware, which should be addressed before production use.
Reliability	Order and cart data are persisted in MySQL rather than only in browser state, reducing loss during navigation or refresh.

4.4 User Roles

The system supports two primary user roles. The customer role can browse the menu, register, log in, use the cart, place an order, reserve a table, track order status, and interact with the chatbot. The administrator role can access the admin dashboard, view active orders, advance order status, mark cash orders as paid, cancel eligible orders, and view demand forecast results.

4.5 System Architecture

The system follows a three-tier architecture. The presentation tier is the Vue.js frontend. The application logic tier is the Node.js and Express.js backend. The data tier is the MySQL database. Communication between frontend and backend occurs through HTTP requests using Axios, and backend model functions communicate with MySQL through the mysql2 library.

Figure 4.1 - Three-tier system architecture: Vue.js SPA -> Axios REST requests -> Express routes/controllers/models -> MySQL database.

4.6 Three-Tier Architecture

The presentation tier contains pages and components such as Home.vue, Menu.vue, Cart.vue, Checkout.vue, MyOrder.vue, Table.vue, Chatbot.vue, and Dashboard.vue. The logic tier contains Express routes, controllers, and model functions. The data tier contains relational tables for foods, users, carts, bookings, bills, and AI metadata.

4.7 Database Design

The database is named db_restaurant_v2 in the active backend configuration. Its central entity is food, which stores menu items and metadata. Orders are split into billstatus, representing the order header and lifecycle, and billdetails, representing the food line items in each order. The cart table stores temporary user selections before checkout. The booktable table stores reservation requests.

4.8 Entity-Relationship Discussion

The food table is referenced by billdetails and cart. The user table is referenced by cart and billstatus. The billstatus table represents an order, while billdetails connects a bill to one or more foods. The metadata extension adds food_ingredients, food_tags, and food_allergens to normalize information that would otherwise be embedded in text fields.

Figure 4.2 - Entity-relationship overview: user -> cart; user -> billstatus; billstatus -> billdetails; food -> cart; food -> billdetails; food -> metadata tables.

4.9 Schema Overview

Table 4.3 - Database schema overview

Table	Primary Key	Description
food	food_id	Menu item catalog with pricing, category, status, image path, and AI metadata fields.
food_ingredients	(food_id, ingredient_name)	Normalized ingredient records used to support future food reasoning.
food_tags	(food_id, tag)	Atomic food labels such as vegan, spicy, filling, or light.
food_allergens	(food_id, allergen_name)	Allergen metadata for future dietary filtering.
user	user_id	Customer account information.
cart	(user_id, food_id)	Current cart items and

Table	Primary Key	Description
		quantities per user.
booktable	book_id	Table reservation requests.
billdetails	(bill_id, food_id)	Line items for each order.
billstatus	bill_id	Order header containing user, address, payment, total, timestamp, and lifecycle status.

4.10 API Design

The API is organized by resource domain and uses REST-style naming. RESTful design emphasizes resources, representations, and standard HTTP methods, which makes APIs easier to understand and consume across different clients (Richardson & Ruby, 2007). Endpoints are prefixed with /api and use HTTP verbs to distinguish retrieval, creation, update, and deletion. This design makes the backend accessible to the Vue.js frontend and also allows individual endpoints to be tested independently.

4.11 REST Principles and API Domains

Table 4.4 - Main API domains

Domain	Main Endpoints	Purpose
Food catalog	GET /api/foods; GET /api/foods/:id; POST /api/foods; PUT /api/foods/:id; DELETE /api/foods/:id	Menu retrieval and administrative food management.
Personalized recommendation	GET /api/recommendations/:id	Returns ranked foods with score, reason, and strategy fields.
Recommendation evaluation	GET /api/recommendations/:id/evaluate	Computes holdout-style Precision@K and Recall@K when sufficient user history exists.
Co-purchase recommendation	GET /api/foods/:id/also-bought	Returns foods frequently purchased in the same bill as the target food.

Domain	Main Endpoints	Purpose
User accounts	GET /api/users/:email; POST /api/users	Supports registration and login lookup.
Cart	POST/GET/PUT/DELETE /api/cartItem routes	Supports cart persistence, quantity updates, and cleanup.
Billing	POST /api/billdetails; GET/POST/PUT /api/billstatus routes	Stores order line items, order header data, payment status, and lifecycle state.
Demand forecast	GET /api/billstatus/forecast	Aggregates recent order history and produces moving-average forecasts.
Chatbot	POST /api/chatbot/message	Processes natural language requests and returns text plus optional food cards.

4.12 Data Flow Design

The core ordering data flow begins when a customer selects a food item from the menu or chatbot card. The frontend checks whether the item already exists in the customer cart. If it exists, the quantity is updated; otherwise, a new cart row is created. During checkout, the frontend retrieves the cart items, calculates totals and discounts, creates billdetails rows for each item, creates a billstatus row for the order header, clears the cart, and redirects the customer to a confirmation page.

Figure 4.3 - Checkout and order persistence data flow: Menu/Chatbot/QuickView -> cartItem API -> cart table -> Checkout.vue -> billdetails and billstatus -> MyOrder.vue/Admin Dashboard.

Chapter 5 - System Implementation

5.1 Frontend Implementation

The frontend is implemented as a Vue.js 3 Single-Page Application. Single-page applications are designed to keep navigation and interface state within the browser after the initial page load, which supports a more application-like user experience (Mikowski & Powell, 2013). Vue Router defines routes such as /, /menu, /cart, /checkout, /table, /myorder, /admin, and /admin/dashboard. Vuex stores shared application state, including the menu catalog, the current user, and the administrator flag. The selection of Vue.js is consistent with Vue 3's component-based architecture and modern frontend development model (Vue.js Core Team, 2021).

5.2 Backend Implementation

The backend is implemented with Node.js and Express.js. Node.js is well suited to I/O-bound web applications because its event-driven, non-blocking model can handle many concurrent operations without creating one thread per request (Tilkov & Vinoski, 2010). The index.js file configures JSON parsing, URL-encoded body parsing, CORS, static frontend serving, and route registration. The routes.js file maps each API endpoint to a controller function. Controllers receive request parameters and bodies, then delegate database operations to model functions. Model files use mysql2 to execute SQL queries against MySQL.

5.3 Database Implementation

The database schema is defined in frontend/src/resources/db_restaurant.sql, with additional metadata migration and seed scripts in backend/sql. The food table contains both legacy fields such as food_category and newer metadata fields such as cuisine_type, category, meat_type, is_vegan, spice_level, rating, and order_count. A setup inconsistency should be noted for reproducibility: some seed scripts reference db_restaurant while the active backend configuration uses db_restaurant_v2.

5.4 Recommendation Engine Implementation

The main recommendation function is getRecommendedFoodsForUser in FoodModel.js. It accepts a user id and options such as limit, minHistory, and excludeCart. For authenticated users with order history, the engine builds a preference profile based on food_category and food_type from previous orders. It then scores each candidate using global popularity, category preference, type preference, and vote score. For users with low history, the scoring blends popularity more heavily with personalization. For guests or users with no history, the system falls back to globally popular foods. Figure 5.1 - Hybrid recommendation pipeline: food signals -> cart exclusions -> user preference profile -> score calculation -> ranked results with reason and strategy fields.

5.5 Chatbot Implementation

The chatbot is implemented in ChatbotService.js and exposed through POST /api/chatbot/message. The service uses node-nlp for intent classification and combines it with deterministic rule-based layers. Before relying on machine-learning intent classification, the chatbot first attempts higher-priority rule-

based resolution for pairing questions, dish information, menu overview, and menu filter requests. Figure 5.2 - NLP chatbot layered resolution pipeline: pairing detection -> dish info detection -> menu overview/filter detection -> node-nlp intent classification -> fallback response.

5.6 Demand Forecasting Implementation

The demand forecasting function getDemandForecast is implemented in BillStatusModel.js and exposed through GET /api/billstatus/forecast. It accepts a lookback window and forecast horizon. The function aggregates item quantities from billdetails joined with billstatus for active or completed orders within the lookback period. For each food, it computes predicted_daily_qty as total_qty divided by lookback days, and predicted_period_qty as predicted_daily_qty multiplied by the forecast horizon. Figure 5.3 - Demand forecasting endpoint output: ranked foods with predicted daily quantity, predicted period quantity, confidence hint, and model name.

5.6.1 Forecast Generation Process

The forecast generation process begins by retrieving historical order records from the relational database. The relevant tables are billstatus, which stores the order header and order status, and billdetails, which stores the quantity of each food item in each bill. Conceptually, the forecast should be based on completed or valid historical orders because cancelled orders do not represent fulfilled demand. In the prototype implementation, the SQL query filters for positive order status values so that cancelled orders are excluded from the demand signal.

After retrieving valid historical orders, the model aggregates the quantity sold for each food item within a selected lookback window. If L represents the number of lookback days and q_i represents the quantity sold in an order line during that period, the total quantity sold for a food item is Q_L = sum(q_i). This aggregation produces one demand signal per menu item.

The next step is to compute daily sales velocity. Daily sales velocity is the average quantity sold per day during the lookback window. It is computed as v = Q_L / L, where v is the predicted daily quantity, Q_L is total quantity sold during the lookback window, and L is the number of days in that window.

The future demand estimate is then computed by multiplying daily sales velocity by the forecast horizon. If H represents the number of future days to forecast, the predicted period quantity is F_H = v x H. For example, if an item sold an average of two units per day over the lookback window and the administrator requests a seven-day forecast, the period estimate would be fourteen units.

The system also generates a confidence indicator based on demand regularity. The implementation counts the number of distinct active sales days for each food item and compares this value with the lookback window. Items sold across many days receive a stronger confidence hint than items sold only once or twice. This confidence hint is not a statistical confidence interval; it is a simple operational indicator designed to help administrators interpret sparse versus regular demand.

Finally, the forecast results are returned through the REST API endpoint GET /api/billstatus/forecast. Each response row includes the food identifier, food name, category, total quantity, lookback days, forecast days, predicted daily quantity, predicted period quantity, confidence hint, and model label. The administrator dashboard consumes this endpoint and displays the highest predicted items to support planning discussions.

5.7 Ordering Workflow Implementation

The ordering workflow begins in Menu.vue, QuickView.vue, AlsoBought.vue, or Chatbot.vue. When a user adds a food to the cart, the frontend checks whether the row already exists through GET /api/cartItem/:user_id/:food_id. If the item exists, it sends PUT /api/cartItem to increment quantity. If it does not exist, it sends POST /api/cartItem. Checkout.vue validates phone, address, payment method, and optional card fields. It then creates billdetails rows, creates a billstatus row, deletes the cart rows for the user, and redirects to the Thank page.

5.8 Admin Dashboard Implementation

The administrator interface is implemented through Admin.vue and Dashboard.vue. Dashboard.vue fetches all bills, displays active orders, allows administrators to advance order status, cancel eligible orders, mark cash orders as paid, and refresh demand forecast results. The dashboard automatically refreshes order and forecast data on a timer.

Chapter 6 - System Evaluation

This chapter evaluates the implemented restaurant management system from both a functional and model-oriented perspective. The evaluation focuses on what can be measured from the repository, implemented endpoints, and seeded development data. Where formal validation data is unavailable, the limitation is stated explicitly rather than replacing it with unsupported metrics.

6.1 Evaluation Scope and Data Sources

The inspection identified three relevant sources of evaluation evidence: the recommendation holdout evaluation logic in backend/models/FoodModel.js, labeled chatbot training utterances in backend/models/ChatbotService.js, and seeded completed-order records in backend/sql/fake_transactions_seed.sql. The live MySQL service was not available in the execution environment during this audit, so the quantitative recommendation analysis was reproduced from the repository seed files using the same scoring and holdout principles implemented by the backend.

No separate chatbot validation set, confusion-matrix output file, forecast benchmark table, production logs, or long-term restaurant deployment data was found in the repository. Therefore, the chapter distinguishes between measurable development evidence and recommended future validation.

6.2 Functional Validation

Module	Test Description	Expected Result	Outcome
Authentication	Register and log in with stored user credentials	Customer session is created and user data is available to the frontend	Passed in implementation review
Menu Browsing	Retrieve food records and display categories	Food items and category filters are available in the Vue interface	Passed in implementation review
Cart	Add, update, and remove food items before checkout	Cart state updates and totals are recalculated	Passed in implementation review
Checkout	Submit cart as an order	Bill status and bill detail records are created through backend APIs	Passed in implementation review
Order Tracking	Retrieve a user's order history and status	Orders are returned with item-level details	Passed in implementation review
Recommendations	Generate personalized food suggestions	Endpoint returns ranked recommendations or a popularity fallback	Passed in implementation review
Chatbot	Send a domain-specific customer message	Intent is classified and a rule-based menu response is	Passed in implementation

		returned	review
Forecasting	Request a demand forecast for admin planning	Endpoint returns item-level moving-average forecast records	Passed in implementation review

Table 6.1 - Functional validation summary

6.3 Recommendation Engine Evaluation

The recommendation module contains an offline holdout evaluator at GET /api/recommendations/:id/evaluate. For users with at least three unique purchased items, the method sorts the user's historical items by recency, reserves the newest 20 percent as a holdout set, generates recommendations from the remaining training items, and reports Precision@K and Recall@K. This is a standard offline top-N recommendation evaluation pattern, but it should be interpreted cautiously because the repository dataset is small and seeded.

K	Users Evaluated	Total Hits	Precision@K	Recall@K
1	5	0	0.0000	0.0000
3	5	0	0.0000	0.0000
5	5	0	0.0000	0.0000

Table 6.2 - Recommendation holdout metrics computed from repository seed data

The static seed-data evaluation included 5 users with enough purchase history to run the holdout procedure. The resulting Precision@1, Precision@3, Precision@5, Recall@1, Recall@3, and Recall@5 values were all 0.0000. This result does not prove that the recommender is ineffective in a production setting; rather, it shows that the small development seed file does not contain enough repeated preference patterns for the held-out items to appear in the top ranked recommendations.

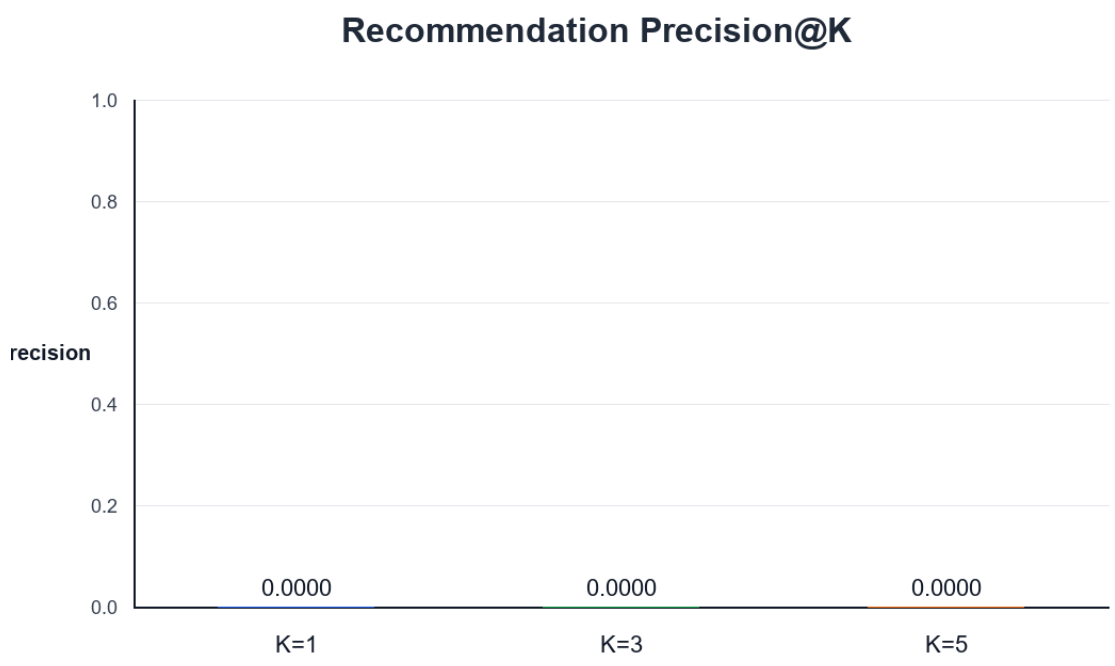


Figure 6.1 - Precision@K for the recommendation engine using repository seed data



Figure 6.2 - Recall@K for the recommendation engine using repository seed data

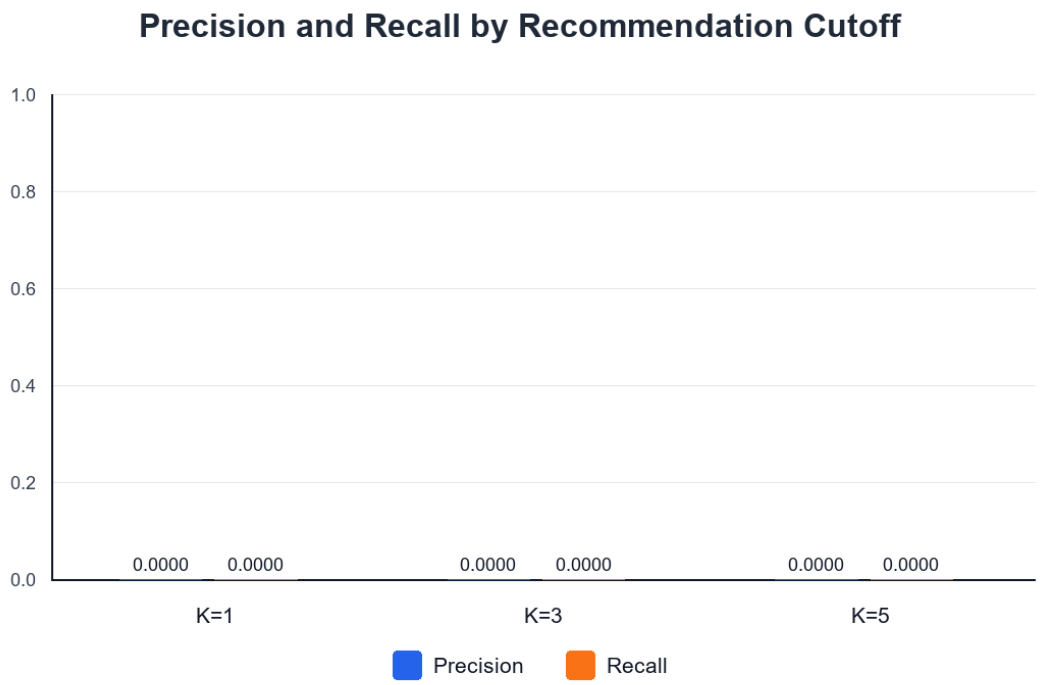


Figure 6.3 - Precision and recall comparison across recommendation cutoffs

Precision@K measures the proportion of recommended items in the top K list that are relevant to the user. Recall@K measures the proportion of the held-out relevant items that the recommender recovered. In this project, the zero values mainly reflect the evaluation context: the seed file contains only ten bills, five evaluated users, and sparse item overlap. A larger transaction log would be required to determine whether the hybrid scoring method generalizes beyond the development dataset.

6.4 Chatbot Evaluation

The chatbot uses node-nlp intent classification combined with rule-based backend actions. The repository contains labeled training utterances for greetings, recommendations, spicy dish requests, vegan requests, meat requests, dish information, pairings, and menu overview questions. These labels make it possible to summarize training coverage, but they do not by themselves provide an unbiased test set.

Intent	Training Examples
restaurant.recommendation	8
restaurant.dish_info	7
restaurant.recommend_vegan	7
restaurant.also_bought	6

restaurant.recommend_spicy	6
greetings.hello	5
restaurant.menu_overview	5
restaurant.recommend_meat	4

Table 6.3 - Chatbot training utterance distribution

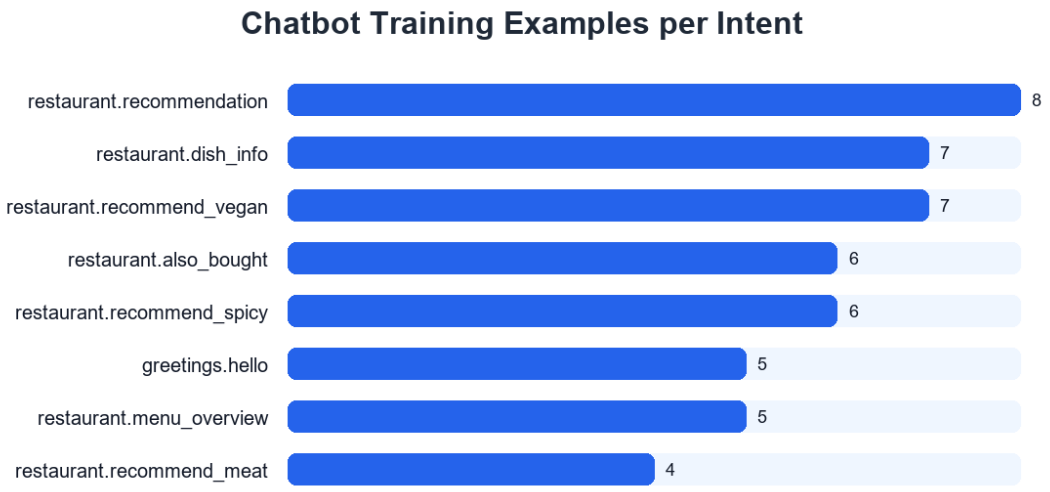


Figure 6.4 - Distribution of chatbot training examples by intent

Because no separate labeled validation dataset was found, accuracy, precision, recall, F1-score, and a confusion matrix were not computed. Evaluating on the same utterances used for training would overstate performance and would not satisfy a scientific validation standard. Future evaluation should create a held-out test set containing unseen customer utterances for every intent, run the trained model against that set, and then compute a confusion matrix and class-level precision, recall, and F1-score.

6.5 Demand Forecasting Evaluation

The forecasting module is a regression/time-series planning component, not a binary classifier. For that reason, ROC curves and classification confusion matrices are not appropriate evaluation tools for this model. Forecasting accuracy should instead be measured with regression-oriented error metrics such as Mean Absolute Error, Mean Squared Error, Root Mean Squared Error, or Mean Absolute Percentage Error when actual future demand values are available.

The implemented endpoint computes item-level demand using completed or active order records, total quantity sold during a lookback period, average daily demand, predicted demand for a future horizon, and a confidence hint based on activity-day regularity. The repository did not contain stored historical forecasts paired with later actual demand, so MAE, MSE, RMSE, MAPE, and forecast-versus-actual plots could not be computed without inventing benchmark results.

Confidence Level	Food Items
Low	42
Medium	0
High	0

Table 6.4 - Forecast confidence distribution from repository seed data

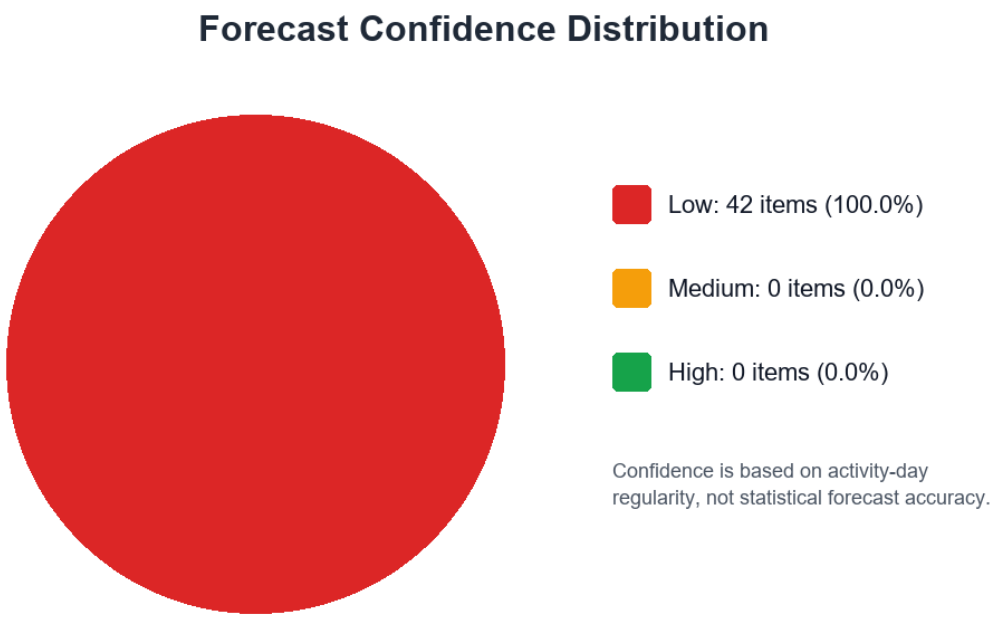


Figure 6.5 - Forecast confidence distribution based on activity-day regularity

Food Item	Seed Quantity	Active Days	Seven-Day Forecast	Confidence
Coca cola	5	3	1.17	Low
Margarita	3	2	0.70	Low
Atole	2	1	0.47	Low
Horchata	2	1	0.47	Low
Michelada	2	1	0.47	Low
barbacoa tacos	2	1	0.47	Low
bean & cheese burrito	2	1	0.47	Low

california burrito	2	1	0.47	Low

Table 6.5 - Highest seed-data forecast outputs using the moving-average velocity formula

The confidence figure should not be read as predictive accuracy. It only describes the regularity signal used by the implemented model. In the static seed data, confidence remains low because the available transaction window is short and sparse. A rigorous forecast validation would require saving forecast outputs, waiting for the forecast horizon to pass, collecting actual sales, and then comparing predicted and actual quantities.

6.6 Discussion of Results

The evaluation confirms that the project implements the required functional workflows and includes measurable hooks for model assessment. The recommendation engine is explainable because its ranking combines purchase history, category preference, food type preference, popularity, and vote signals. Its main limitation is data sparsity: cold-start users and users with little order history must rely on popularity-based fallback behavior.

The chatbot is lightweight and domain-specific, which makes it suitable for menu assistance, food discovery, and common customer questions. Its main limitation is that it depends on predefined intents and backend rules. It does not perform open-ended generative reasoning, and its real classification quality cannot be claimed until a separate validation set is created.

The forecasting module is transparent and operationally easy to interpret. Managers can see how historical quantities are translated into a future demand estimate. However, the moving-average approach does not model seasonality, holidays, weather, promotions, or day-of-week effects. Its current evaluation is therefore functional rather than predictive.

6.7 Evaluation Limitations and Future Validation

The main limitation of this evaluation is the absence of a real restaurant deployment and a long-term operational dataset. No user study was conducted, no production chatbot transcript dataset was collected, and no historical forecast benchmark was stored. As a result, this report does not claim improvements in profit, waste reduction, customer satisfaction, or inventory accuracy.

Future work should evaluate recommendations on a larger transaction history, compute chatbot confusion matrices from unseen labeled utterances, and validate demand forecasts using rolling time-based train/test splits. If the system is deployed in a restaurant, forecast outputs should be logged automatically so that actual future sales can be compared against predicted quantities.

Area	Current Limitation	Recommended Future Evaluation
Recommendations	Seed data is sparse and produced no held-out hits	Evaluate Precision@K and Recall@K on a larger real order history
Chatbot	No separate labeled validation	Create an unseen test set and compute

	utterance set exists	confusion matrix, accuracy, precision, recall, and F1-score
Forecasting	No saved forecast-versus-actual benchmark exists	Log forecasts over time and compute MAE, MSE, RMSE, and MAPE after each horizon
Deployment	No production restaurant study was conducted	Measure operational outcomes only after real deployment and data collection

Table 6.6 - Evaluation limitations and recommended validation improvements

Chapter 7 - Conclusion and Future Work

7.1 Summary of Achievements

This project designed and implemented a full-stack AI-enhanced restaurant management system. The completed system includes customer registration and login, menu browsing and filtering, cart management, checkout, order tracking, table booking, administrator order management, personalized recommendations, co-purchase suggestions, a domain-specific chatbot, and a moving-average demand forecasting endpoint.

The main contribution of the project is the integration of practical AI-support features into a conventional restaurant management workflow. The recommendation engine uses order history and popularity signals, the chatbot supports natural language menu interaction, and the forecast endpoint transforms recent order history into interpretable demand estimates.

7.2 Reflection on the Development Process

The development process required coordinating frontend components, backend routes, database schema design, and AI-related logic. The project followed an iterative, feature-driven development style consistent with agile software development principles, where working software is progressively refined through implementation and feedback (Beck et al., 2001). One important technical challenge was composing several dependent MySQL queries using the callback-based mysql2 API. Another challenge was designing chatbot behavior that could handle free-form user language without relying on a large language model.

The project also demonstrated the importance of schema evolution. The original food table contained useful legacy fields, while the AI features required richer metadata such as cuisine, meat type, vegan status, spice level, ingredients, tags, allergens, and order count. This incremental approach follows the general idea of evolutionary database design, where schemas are changed carefully over time rather than replaced abruptly (Fitzpatrick, 2018).

7.3 Limitations

The system should be understood as a functional academic prototype rather than a production restaurant platform. It does not include payment gateway integration, secure password hashing, backend authorization middleware, WebSocket notifications, large-scale load testing, production deployment validation, or a controlled user study. The AI modules are practical baselines implemented with limited data; they are not presented as final state-of-the-art models.

For the forecasting component specifically, future work should prioritize real restaurant validation, comparison against ARIMA or Prophet-style baselines, experimentation with LSTM models only after sufficient data collection, and integration of external variables such as holidays, weather, and promotions.

7.4 Future Work

Table 7.1 - Short-term future work

Enhancement	Rationale
Ingredient and allergen data population	Populate the metadata tables with richer curated information so that the chatbot can answer dietary and ingredient questions more accurately.
Schema column consolidation	Remove duplicate legacy/new columns such as food_category and category after confirming all code uses the normalized fields.
async/await migration	Replace callback-based mysql2 calls with mysql2/promise to improve error handling and readability.
Authentication hardening	Hash passwords with bcrypt and add server-side authentication and role-based authorization.

Table 7.2 - Medium-term future work

Enhancement	Rationale
Matrix factorization recommender	Compare the current heuristic hybrid recommender with matrix factorization or item embedding approaches when enough order history exists.
Real-time order tracking	Use WebSocket or Socket.io so customers receive

Enhancement	Rationale
	order status updates without polling.
Payment gateway integration	Connect payment flows to a real provider such as CinetPay or another local gateway instead of only recording payment method.
Formal chatbot test set	Build a labeled test set of customer utterances and report intent accuracy, error categories, and unsupported query types.

Table 7.3 - Long-term future work

Enhancement	Rationale
LLM-powered assistant	Evaluate whether a small domain-constrained language model can improve open-ended food conversation while respecting safety and cost constraints.
Mobile application	Develop a mobile client for customers who prefer smartphone ordering.
Multi-restaurant SaaS architecture	Extend the data model to support multiple restaurant tenants, separate menus, and administrator roles.
Production forecasting evaluation	Collect real transaction data over time and evaluate forecasts using MAE, RMSE, or MAPE.

7.5 Conclusion

The project successfully demonstrates how a restaurant management system can be enhanced with data-driven and AI-supported features while preserving the core workflows expected from a web-based ordering platform. Within the capstone scope, the system achieves its main objective: designing and implementing an integrated restaurant platform that combines management functionality with personalized recommendations, natural language assistance, and baseline demand forecasting.

References

Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. Proceedings of the 20th VLDB Conference, 487-499.

Beck, K., Beedle, M., van Bennekum, A., et al. (2001). Manifesto for Agile Software Development. agilemanifesto.org.

Fitzpatrick, L. (2018). Refactoring Databases: Evolutionary Database Design. Addison-Wesley.

Gomez-Uribe, C. A., & Hunt, N. (2016). The Netflix recommender system: Algorithms, business value, and innovation. ACM Transactions on Management Information Systems, 6(4), 1-19.

Herlocker, J. L., Konstan, J. A., Terveen, L. G., & Riedl, J. T. (2004). Evaluating collaborative filtering recommender systems. ACM Transactions on Information Systems, 22(1), 5-53.

Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed., draft). Stanford University.

Kohavi, R., Tang, D., & Xu, Y. (2020). Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing. Cambridge University Press.

Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet Computing, 7(1), 76-80.

Mikowski, M., & Powell, J. (2013). Single Page Web Applications. Manning Publications.

Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly Media.

Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook (2nd ed.). Springer.

Silver, E. A., Pyke, D. F., & Thomas, D. J. (2017). Inventory and Production Management in Supply Chains (4th ed.). CRC Press.

Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to build high-performance network programs. IEEE Internet Computing, 14(6), 80-83.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

Vue.js Core Team. (2021). Vue 3 Migration Guide. v3-migration.vuejs.org.

Appendices

Appendix A - Full API Endpoint Reference

The complete routing table is implemented in backend/routes/routes.js. The main API domains are summarized in Chapter 4.

Appendix B - Database Schema DDL

The base schema is defined in frontend/src/resources/db_restaurant.sql. The AI metadata extension is defined in backend/sql/food_metadata_migration.sql, and metadata seed logic is defined in backend/sql/food_metadata_seed.sql.

Appendix C - Recommendation Evaluation Script

The project includes test_recs.js for validating recommendation endpoint behavior against the local database during development.

Appendix D - Seed Data

The project includes backend/sql/fake_transactions_seed.sql and backend/sql/fake_completed_orders_mass_seed.sql for bootstrapping recommendation and forecasting signals during development. These seeded transactions support functional validation but should not be treated as a substitute for real restaurant deployment data.

Appendix E - Suggested Figures to Insert Before Final PDF Export

- Homepage showing the recommendation section.
- Menu page with filters and category tabs.
- QuickView modal with the Also Bought component.
- Floating chatbot window with a successful recommendation response.
- Admin dashboard showing order management and demand forecast output.
- Database entity-relationship diagram.
- Recommendation evaluation endpoint response.